



Skills For Real Engineers

Complete Instructional Reference

source repository	https://github.com/mattpocock/skills
source talk	https://youtu.be/v4F1gFy-hqg
author of skills	Matt Pocock
license	MIT
plugin version	1.2.3
document type	instructional_reference
skill count	35
document revision	1

0. About This Document

0.1 What this is

This is a complete instructional reference for the agent skills published at github.com/mattpocock/skills. It covers the reasoning behind the collection, the vocabulary the skills share, the flows that connect them, and a uniform record for every one of the 35 skills in the repository.

Two files exist with identical content. The PDF is set for human reading. The Markdown is set for machine parsing. Nothing in one is missing from the other.

0.2 How to read it

If you want	Read
The argument for why any of this matters	Section 1
A map of the repository	Section 2
To install and configure	Section 3
The shared vocabulary	Section 4
The order to run skills in	Section 5
One specific skill	Section 6
The files a skill loads on demand	Section 7
The files the skills write into your repo	Section 8
Worked sequences for common situations	Section 9
Books the skills draw on	Section 11

0.3 Notation

- `/name` means a skill invoked by typing it into an agent session.
 - **User-invoked** means only a human can start it.
 - **Model-invoked** means the agent can also reach for it on its own.
 - File paths are relative to the repository root.
-

1. The Argument

1.1 Thesis

Software fundamentals matter more now than they did before coding agents existed, not less. This claim comes from Matt Pocock, who built the skills in this repository while teaching a course on using coding agents for production work. The talk that sets out the argument is linked in Section 12.

1.2 The failure of specification-only development

One popular approach says: write a specification, generate code from it, and never look at the code again. When something is wrong, edit the specification and generate again. Treat the code as disposable output.

Run that loop repeatedly and the code degrades. The first pass produces something workable. Each further pass produces something worse. The reason is that every regeneration optimises for the change in front of it and ignores the design of the whole system.

Two ideas from older literature name what happens.

Complexity, in John Ousterhout's definition, is anything about the structure of a system that makes it hard to understand and modify. A bad codebase is one that resists change. A good codebase is one you can change without breaking things.

Software entropy, described in *The Pragmatic Programmer*, is the tendency of a system to decay when each change is made without regard for the whole. Agents accelerate this because they produce change faster than any human team can.

1.3 Code is not cheap

The slogan that drives specification-only development is that code has become cheap. The counter-argument is that bad code has never been more expensive. An agent working in a clean codebase is fast and accurate. The same agent in a tangled codebase produces guesses. So the quality of the codebase now sets a ceiling on how much value an agent can deliver. Good codebases matter more than they used to, which means engineering fundamentals matter more than they used to.

1.4 Four failure modes and their fixes

#	Failure mode	Root cause	Fix	Skills
1	The agent did not build what I wanted	No shared design concept between human and agent	Interview before building	<code>/grill-me</code> , <code>/grill-with-docs</code> , <code>/grilling</code>
2	The agent is far too verbose	No shared language for the domain	Build a glossary the agent reads	<code>/domain-modeling</code> , <code>CONTEXT.md</code> , <code>/wait-what</code>

#	Failure mode	Root cause	Fix	Skills
3	The code does not work	Weak or unused feedback loops	Small steps against a tight loop	<code>/tdd</code> , <code>/diagnosing-bugs</code> , <code>/code-review</code>
4	We built a ball of mud	No investment in design	Deep modules and honest seams	<code>/codebase-design</code> , <code>/improve-codebase-architecture</code>

1.5 Failure mode 1 in detail

Nobody knows exactly what they want. That is the standard condition of software work, not a personal failing. The gap between what you pictured and what the agent produced is a requirements-gathering gap.

Frederick Brooks called the shared, unwritten idea held between collaborators the **design concept**. It is not a document. It is the theory of the thing being built. When you and the agent do not share one, the agent guesses.

The fix is an interview. The agent questions you until the design concept is shared. Sixty questions is normal. The conversation that results becomes the specification.

This is deliberately different from a default planning mode, which tends to rush toward producing a plan document. Reaching shared understanding comes first, and the artifact comes after.

1.6 Failure mode 2 in detail

A developer dropped into an unfamiliar domain uses twenty words where a specialist uses one, because the specialist has precise terms and the newcomer does not. Agents are permanently in that position: dropped into a project, inferring jargon as they go.

Domain-driven design calls the fix a **ubiquitous language**: one vocabulary shared by conversation, code, and documentation, all derived from the same model of the domain. In these skills it lives in a file called `CONTEXT.md`.

The gains compound:

- Planning conversations get shorter and sharper.
- Variables, functions, and files get named consistently.
- The codebase becomes easier for the agent to navigate.
- The agent spends fewer tokens thinking, because the concepts have names.
- Implementations match plans more often, because the plan is unambiguous.

1.7 Failure mode 3 in detail

Feedback loops are the usual set: static types, browser access for front-end work, and automated tests. Having them is not enough. Agents tend to write a large amount of code and only then check any of it.

The Pragmatic Programmer calls this outrunning your headlights. The rate of feedback is the speed limit. Small deliberate steps are the corrective, and test-driven development is the discipline that enforces them: write a failing test, write only enough code to pass it, then attend to the design.

Testing is hard because the decisions are interdependent. How large is the unit under test. What gets mocked. Which behaviours are worth testing at all. The size of the unit affects flakiness, which affects how many behaviours are worth asserting, which affects what has to be mocked.

This is where the argument closes its loop: a testable codebase is a well-designed codebase, and a well-designed codebase gives the agent better feedback, which produces better code.

1.8 Failure mode 4 in detail

Ousterhout distinguishes deep modules from shallow ones.

A **deep module** holds a large amount of behaviour behind a small interface. You may look inside, but you do not need to. A **shallow module** holds little behaviour behind an interface nearly as complicated as the implementation.

A codebase made of many shallow modules is a field of small blobs. An agent exploring it has to walk through many files, hold many dependencies in context, and frequently fails to assemble an accurate picture. Agents are also very good at producing exactly this shape of codebase if left alone.

The same code arranged into fewer, deeper modules gives the agent a small number of interfaces to learn. You design the interfaces carefully. You can leave much of the implementation to the agent.

This also protects human attention. Shipping faster than before is tiring because you still have to hold the system in your head. Deep modules let you treat parts of the system as grey boxes: design the interface, delegate the implementation, verify through the boundary. Reserve full review for the parts that warrant it, such as money, security, and data integrity.

The discipline that follows is Kent Beck's: invest in the design of the system every day. Specification-only development divests from design. This collection invests in it at every step.

1.9 The role you keep

Treat the agent as an excellent tactical programmer working at ground level. Someone still has to work at the strategic level, deciding what the system should look like and holding the line on its design. That is the human role, and it runs on fundamentals that predate agents by decades.

2. Repository Map

2.1 Identity

Field	Value
Repository	<code>github.com/mattpocock/skills</code>
Plugin name	<code>mattpocock-skills</code>
Plugin version	<code>1.2.3</code>
Author	Matt Pocock
License	MIT
Marketplace owner	<code>mattpocock</code>
Newsletter	<code>aihero.dev/s/skills-newsletter</code>

2.2 Design principles of the collection

- Small units rather than a framework that owns your process.
- Composable, so one skill can call another.
- Model-agnostic, so they work across agent harnesses.
- Editable, so you can fork and adapt them.
- Grounded in established engineering literature rather than novelty.

The stated contrast is with process frameworks that take control of the workflow. When a framework owns the process, bugs in the process become hard to reach. These skills stay small so you keep control.

2.3 Directory structure

skills/	
engineering/	18 skills for code work
productivity/	7 skills for general workflow
misc/	4 occasional tools, not in the plugin
in-progress/	6 beta skills, not in the plugin
deprecated/	empty by policy
docs/	published documentation pages
scripts/	maintainer scripts
.claude-plugin/	plugin and marketplace manifests
.agents/	repository conventions and decision records
.out-of-scope/	recorded rejections with reasons
.changeset/	release note fragments
CLAUDE.md	repository instructions for agents
AGENTS.md	symlink to CLAUDE.md
CONTEXT.md	the repository's own glossary
README.md	project readme
CHANGELOG.md	release history
LICENSE	MIT

2.4 Skill counts

Bucket	Skills	Included in plugin
engineering	18	Yes
productivity	7	Yes
misc	4	No
in-progress	6	No
deprecated	0	No
Total	35	25 in plugin

2.5 Anatomy of a skill folder

<skill-name>/	
SKILL.md	the skill itself, with YAML frontmatter
agents/openai.yaml	invocation policy for Codex-style harnesses
<REFERENCE>.md	optional files loaded only when needed
scripts/ or *.sh	optional templates and helper scripts

Frontmatter fields in use:

Field	Meaning
name	The skill identifier, which becomes /name
description	The trigger text the agent reads to decide relevance
disable-model-invocation	true marks the skill user-invoked only
argument-hint	Prompt text for the argument the skill expects

2.6 Invocation model

User-invoked skills are reachable only by typing them. Their job is to orchestrate. In Claude Code they carry `disable-model-invocation: true`. In Codex the equivalent is `policy.allow_implicit_invocation: false` in `agents/openai.yaml`.

Model-invoked skills can be typed by you or reached for by the agent when the task fits. They hold reusable discipline.

The composition rule: a user-invoked skill may call model-invoked skills, but never another user-invoked skill.

3. Installation

3.1 Choose one route

Two routes exist and they represent two philosophies. Install one only. Installing both gives you every skill twice.

Route	What you get	Updates	Editable
Claude Code plugin	A managed, read-only bundle of 25 skills	Automatic	No
skills installer	Editable files copied into your project	On request	Yes

3.2 Route A: Claude Code plugin

```
claude plugins install mattpocock-skills
```

From inside a session:

```
/plugin install mattpocock-skills
```

The plugin sits in the official marketplace, so no marketplace has to be added first. This route excludes the `misc` and `in-progress` buckets.

3.3 Route B: the installer, for Codex and other agents

```
npx skills@latest add mattpocock/skills
```

The installer asks which skills to take and which agent harnesses to install them on. Take `setup-matt-pocock-skills`, because the engineering skills depend on the configuration it writes.

Install a single beta skill by name:

```
npx skills@latest add mattpocock/skills --skill=<name>
```

Pull later changes when you want them:

```
npx skills update
```

3.4 Configure each repository

Run this once per repository, before using the other engineering skills:

```
/setup-matt-pocock-skills
```

It asks three questions:

1. Which issue tracker this repository uses.
2. Which label strings map to the five canonical triage roles.
3. Where domain documents should live.

It writes the answers to `docs/agents/`, so every later skill knows whether to call `gh issue create`, write a file under `.scratch/`, or follow some other workflow.

3.5 Verify

```
/ask-matt
```

The router answers with the flow that fits your situation, which also confirms the skills are loaded.

4. Core Vocabulary

The skills use these terms precisely and expect you to do the same. Substituting near-synonyms is the failure this vocabulary exists to prevent.

4.1 Conversation and alignment

Design concept

The shared, unwritten understanding of the thing being built, held jointly by you and the agent. Not an artifact. Named by Frederick Brooks.

Design tree

The structure of a plan, where each decision branches into the decisions that hang off it.

Frontier

Every decision whose prerequisites are already settled. These are the questions answerable now, without guessing at answers not yet heard.

Round

One pass over the whole frontier. Questions are numbered, each carries a recommended answer, and the agent waits for replies before recomputing the frontier.

Grilling

The interview discipline that works the design tree round by round until the frontier is empty.

4.2 Domain and language

Ubiquitous language

One vocabulary shared by conversation, code, and documentation, all derived from the same domain model.

CONTEXT .md

The file holding that vocabulary. A glossary and nothing else. No implementation detail, no specifications, no scratch notes.

CONTEXT-MAP .md

Present only when a repository has multiple bounded contexts. It points at where each context lives.

ADR

Architectural Decision Record. Numbered files in `docs/adr/`. Written only when a decision is hard to reverse, surprising without context, and the result of a real trade-off. If any of the three is missing, skip it.

4.3 Code design

Module

Anything with an interface and an implementation. Deliberately scale-agnostic: a function, a class, a package, or a slice spanning tiers. Do not say unit, component, or service.

Interface

Everything a caller must know to use the module correctly. The type signature, and also invariants, ordering constraints, error modes, required configuration, and performance characteristics. Do not say API or signature, which cover only the type-level surface.

Implementation

What sits inside a module. Distinct from adapter: a thing can be a small adapter with a large implementation, such as a Postgres repository, or a large adapter with a small implementation, such as an in-memory fake.

Depth

Leverage at the interface. How much behaviour a caller or a test can exercise per unit of interface it has to learn. Deep means a large amount of behaviour behind a small interface. Shallow means an interface nearly as complicated as the implementation.

Seam

A place where behaviour can be altered without editing in that place. The location at which a module's interface lives. Michael Feathers coined it. Do not say boundary, which collides with the bounded context of domain-driven design.

Adapter

A concrete thing that satisfies an interface at a seam. Describes the role it fills, not what is inside it.

Leverage

What callers gain from depth. More capability per unit of interface learned. One implementation repays across many call sites and many tests.

Locality

What maintainers gain from depth. Change, bugs, knowledge, and verification concentrate in one place instead of spreading across callers. Fix once, fixed everywhere.

4.4 Design tests

The deletion test

Imagine deleting the module. If complexity vanishes, it was a pass-through. If complexity reappears across many callers, it was earning its keep.

The interface is the test surface

Callers and tests cross the same seam. Wanting to test past the interface means the module is probably the wrong shape.

One adapter is a hypothetical seam, two adapters is a real one

Do not introduce a seam unless something actually varies across it.

Depth is a property of the interface, not the implementation

A deep module may be built internally from small swappable parts. Those parts are internal seams, private to the implementation and usable by its own tests.

4.5 Delivery**Tracer bullet**

A narrow but complete path through every layer, from schema to interface, that can be demonstrated on its own.

Vertical slice

The same idea applied to work breakdown. One slice cuts through all layers. The opposite, horizontal slicing, builds one layer at a time and cannot be verified until the end.

Blocking edge

A declared dependency from one ticket to another that must complete first. A ticket with no blockers can start immediately.

Wide refactor

One mechanical change whose blast radius covers the whole codebase, such as renaming a shared column. It cannot land as a vertical slice. Sequence it as expand, migrate in batches, then contract.

Expand and contract

Add the new form beside the old so nothing breaks, migrate call sites in batches sized by blast radius, then delete the old form once no caller remains.

4.6 Feedback and sessions**Feedback loop**

One command that goes red on the specific problem in front of you. Building it is most of the work of debugging.

Tight loop

Fast, sharp, and deterministic. A two-second deterministic loop is a different tool from a thirty-second flaky one.

Smart zone

The context window within which a model still reasons sharply, roughly 150,000 tokens on current models. Approaching it means compacting at a phase boundary rather than pushing on degraded.

Phase

A chunk of work inside a session, such as the grilling, the implementation, or the review. A phase ends when you think the work is done.

Phase boundary

The gap between two phases. The only place compaction belongs. Compacting mid-phase makes the agent lose the thread.

4.7 Writing for agents

Context pointer

A reference held in context that names out-of-context material and encodes the condition for reaching it. A skill description is one. A line in `AGENTS.md` naming a document is the same object. The wording of the pointer, not its target, decides when the material gets reached.

Context load

The cost of always-loaded material on the agent's window. Spent every turn whether or not it fires.

Cognitive load

The cost on the human of knowing which documents exist and when to reach for each. Not a cost to minimise blindly. It is the price of human agency.

Progressive disclosure

Moving material out of the main file and behind a pointer, so it loads only when needed. Inline what every branch needs. Push out what only some branches reach.

5. Flows

5.1 The main flow: idea to shipped

The route most work travels.

Step	Skill	What happens
1	<code>/grill-with-docs</code>	Sharpen the idea by interview, leaving a paper trail in <code>CONTEXT.md</code> and ADRs. Use <code>/grill-me</code> instead when there is no working directory.
2	<code>/prototype</code>	Only when a question needs a runnable answer. Bridge in and out with <code>/handoff</code> , because a prototype lives in its own directory.
3a	<code>/to-spec</code> then <code>/to-tickets</code>	For a multi-session build. Produces a specification, then tracer-bullet tickets with blocking edges.
3b	<code>/implement</code>	For a single-session build, in the same context window.
4	<code>/implement</code> per ticket	Each ticket in a fresh context. Drives <code>/tdd</code> internally at agreed seams.
5	<code>/code-review</code>	Two-axis review of the diff before committing. Called automatically by <code>/implement</code> .

5.2 Context hygiene

Keep steps 1 to 3 in one unbroken context window. Do not compact or clear until after `/to-tickets`, so the interview, the specification, and the tickets all build on the same thinking. Each `/implement` then starts fresh from its ticket, because each ticket is self-contained and the previous one's context is disposable.

If a session approaches the smart zone before `/to-tickets`, compact at the nearest phase boundary rather than continuing on a degraded window.

5.3 On-ramps

Three starting situations generate work and then merge onto the main flow.

Situation	Entry skill	Notes
Bugs and requests piling up	<code>/triage</code>	Only for issues you did not create. Tickets from <code>/to-tickets</code> are already agent-ready and must not be triaged.
Something is broken	<code>/diagnosing-bugs</code>	For hard bugs, flakes, and regressions. Hands off to <code>/improve-codebase-architecture</code> when the finding is that no good seam exists.
A large, foggy effort	<code>/wayfinder</code>	For work too big for one session, where the route to the destination is not visible yet.

5.4 Underneath everything

Two skills run as vocabulary layers rather than steps: `/codebase-design` supplies the module and seam language, and `/domain-modeling` keeps the domain glossary current. Other skills call them rather than restating their contents.

6. Skill Reference

6.1 Engineering skills

Eighteen skills for code work. All are included in the plugin.

6.1.1 ask-matt

- **Invocation:** `/ask-matt`
 - **Mode:** User-invoked
 - **Path:** `skills/engineering/ask-matt/SKILL.md`
 - **Purpose:** Answer the question of which skill or flow fits the situation in front of you.
 - **Use when:** You do not remember what is available, or you are unsure whether a piece of work belongs in the main flow or an on-ramp.
 - **How it works:** Acts as a router over the user-invoked skills. Describes the main flow from idea to shipped, the three on-ramps, the standalone skills, and the vocabulary layers that run underneath.
 - **Bundled files:** `PHASE-BOUNDARIES.md`, which explains where compaction belongs.
 - **Pairs with:** Everything. It is the index.
-

6.1.2 grill-with-docs

- **Invocation:** `/grill-with-docs`
 - **Mode:** User-invoked
 - **Path:** `skills/engineering/grill-with-docs/SKILL.md`
 - **Purpose:** Run a relentless interview that also builds the project's domain model as it goes.
 - **Use when:** Starting any change inside a working directory. This is the default entry point to the main flow.
 - **How it works:** Composes two skills. It runs the `/grilling` interview and layers `/domain-modeling` on top, so terminology gets sharpened and `CONTEXT.md` and ADRs get updated inline as decisions crystallise.
 - **Outputs:** A shared understanding, plus updates to `CONTEXT.md` and any ADRs warranted by the conversation.
 - **Pairs with:** `/to-spec` and `/to-tickets` downstream. Use `/grill-me` instead when no repository is present to leave a trail in.
-

6.1.3 triage

- **Invocation:** `/triage`
- **Mode:** User-invoked
- **Path:** `skills/engineering/triage/SKILL.md`
- **Purpose:** Move incoming issues and external pull requests through a small state machine.
- **Use when:** Bug reports and feature requests you did not write are piling up.
- **How it works:** Every issue carries exactly one category role and one state role. Categories are `bug` and `enhancement`. States are `needs-triage`, `needs-info`, `ready-for-agent`, `ready-for-human`, and `wontfix`. An unlabelled issue enters at `needs-triage`. From there it moves onward, and `needs-info` returns to `needs-triage` once the reporter replies. Conflicting state roles get flagged to the maintainer

before anything else happens. A pull request is treated as an issue with attached code, so the same machine applies.

- **Disclosure rule:** Every comment or issue posted during triage begins with a visible note that it was generated by AI during triage.
- **Bundled files:** `AGENT-BRIEF.md` on writing durable briefs, `OUT-OF-SCOPE.md` on the rejection knowledge base.
- **Pairs with:** `/implement`, which picks up anything marked `ready-for-agent`.
- **Do not:** Triage tickets produced by `/to-tickets`. They are already agent-ready.

6.1.4 improve-codebase-architecture

- **Invocation:** `/improve-codebase-architecture`
- **Mode:** User-invoked
- **Path:** `skills/engineering/improve-codebase-architecture/SKILL.md`
- **Purpose:** Survey a codebase for deepening opportunities and present them as a visual report.
- **Use when:** Every few days, and whenever debugging reveals there is no good seam to lock a bug down.
- **How it works:** Three phases. First it scopes, using your stated direction or, failing that, the commit history to find the parts of the codebase that keep changing. It reads `CONTEXT.md` and nearby ADRs, then sends a sub-agent to walk the code looking for friction: concepts that require bouncing between many small modules, shallow modules, pure functions extracted only for testability while the real bugs live in how they are called, coupling that leaks across seams, and code that is hard to test through its current interface. It applies the deletion test to anything suspected of being shallow. Second, it writes a self-contained HTML report to the operating system temporary directory, never into the repository, and opens it. Third, once you pick a candidate, it runs `/grilling` over that one choice.
- **Report contents per candidate:** files involved, the problem, the solution in plain language, benefits framed as locality and leverage, a before and after diagram, and a recommendation strength of Strong, Worth exploring, or Speculative. The report closes with a top recommendation.
- **Honest limitation:** It is a survey, not a rescue. On a genuinely old codebase it will find real candidates, but it will not untangle the mud for you.
- **Bundled files:** `HTML-REPORT.md`, the report scaffold and diagram patterns.
- **Pairs with:** `/codebase-design` for vocabulary, `/domain-modeling` for naming.

6.1.5 setup-matt-pocock-skills

- **Invocation:** `/setup-matt-pocock-skills`
- **Mode:** User-invoked
- **Path:** `skills/engineering/setup-matt-pocock-skills/SKILL.md`
- **Purpose:** Write the per-repository configuration the engineering skills assume.

- **Use when:** Once per repository, before first use of the other engineering skills.
 - **How it works:** Explores first and assumes nothing. It reads the git remotes, `AGENTS.md` and `CLAUDE.md`, `CONTEXT.md` and `CONTEXT-MAP.md`, any `docs/adr/` directories, `docs/agents/`, and `.scratch/`. It checks whether the triage skill is even installed, and looks for monorepo signals. Then it presents findings and takes the sections in order, leading each with a recommended answer so you can accept in one word, and skipping sections that exploration already settled.
 - **Sections:** issue tracker, triage label vocabulary, domain document layout.
 - **Tracker options:** GitHub via the `gh` CLI, GitLab via the `glab` CLI, or local markdown under `.scratch/`.
 - **Bundled files:** `issue-tracker-github.md`, `issue-tracker-gitlab.md`, `issue-tracker-local.md`, `triage-labels.md`, `domain.md`.
-

6.1.6 to-spec

- **Invocation:** `/to-spec`
 - **Mode:** User-invoked
 - **Path:** `skills/engineering/to-spec/SKILL.md`
 - **Purpose:** Turn the current conversation into a specification and publish it.
 - **Use when:** The interview is finished and the work spans more than one session.
 - **How it works:** No interview. It synthesises what has already been discussed. First it explores the repository if it has not already, using the domain glossary throughout and respecting nearby ADRs. Then it sketches the seams at which the feature will be tested, preferring existing seams to new ones and proposing new ones at the highest point possible. Fewer seams is better, and one is ideal. It checks the seams with you before writing. Then it writes the specification and publishes it with the `ready-for-agent` label, needing no further triage.
 - **Specification template:** Problem statement from the user's perspective, solution from the user's perspective, a long numbered list of user stories in the form of actor, feature, and benefit, implementation decisions covering modules and interfaces touched, and testing decisions.
 - **Excluded from the specification:** File paths and code snippets, because they go stale. The one exception is a snippet from a prototype that encodes a decision more precisely than prose, such as a state machine or a schema, trimmed to the decision-rich part.
-

6.1.7 to-tickets

- **Invocation:** `/to-tickets`
- **Mode:** User-invoked
- **Path:** `skills/engineering/to-tickets/SKILL.md`
- **Purpose:** Break a plan, specification, or conversation into tracer-bullet tickets with declared blocking edges.
- **Use when:** Immediately after `/to-spec`, in the same context window.

- **How it works:** Five steps. Gather context from the conversation or a passed reference. Explore the codebase if needed, looking for prefactoring that would make the change easy before making the change. Draft vertical slices. Quiz you on the breakdown. Publish to the configured tracker.
 - **Slice rules:** Each slice cuts a narrow but complete path through every layer. A completed slice is demonstrable on its own. Each slice fits in one fresh context window. Prefactoring goes first.
 - **Wide refactors:** The exception to vertical slicing. Sequence them as expand, then migrate in batches sized by blast radius with each batch blocked by the expand, then contract once no caller remains. Where batches cannot stay green alone, keep the sequence but share an integration branch, with green promised only at a final integrate-and-verify ticket.
 - **Publication shape:** On a local tracker, one file per ticket with edges as text. On a real tracker, native blocking links, so any ticket whose blockers are done can be picked up.
-

6.1.8 implement

- **Invocation:** `/implement`
 - **Mode:** User-invoked
 - **Path:** `skills/engineering/implement/SKILL.md`
 - **Purpose:** Build the work described by a specification or a set of tickets.
 - **Use when:** Once the work is specified. Run it per ticket, clearing context between tickets.
 - **How it works:** Drives `/tdd` where possible, at pre-agreed seams. Runs typechecking regularly and single test files regularly, then the full suite once at the end. Closes out by running `/code-review`, then commits to the current branch.
 - **Pairs with:** `/tdd` inside, `/code-review` after.
-

6.1.9 wayfinder

- **Invocation:** `/wayfinder`
- **Mode:** User-invoked
- **Path:** `skills/engineering/wayfinder/SKILL.md`
- **Purpose:** Chart a route through work too large for one session, as a shared map of decision tickets.
- **Use when:** A greenfield project or a very large feature arrives wrapped in fog, where the way to the destination is not visible yet. Not for well-scoped features.
- **How it works:** Naming the destination is the first act, because it shapes every ticket. The map is a single issue labelled `wayfinder:map`, and its tickets are child issues. The map is an index, not a store: each decision lives in exactly one place, its own ticket, and the map only gists and links. Tickets get worked one at a time until nothing is left to decide.
- **Core constraint:** Produce decisions, not deliverables. The pull to start building is usually the signal that the map has reached its edge and it is time to hand off. An effort can override this in its notes.

- **Readability rule:** Refer to every map and ticket by its title, never by a bare number or slug. Identifiers ride inside the name, never in place of it.
 - **Map body:** A destination section, a notes section for domain and standing preferences, and a decisions-so-far index with one line per closed ticket. Open tickets are not listed, because they are found by query.
 - **Cost:** The most cognitively demanding flow in the collection. Slower and denser than `/grill-with-docs`.
-

6.1.10 tdd

- **Invocation:** `/tdd`
 - **Mode:** Model-invoked
 - **Path:** `skills/engineering/tdd/SKILL.md`
 - **Purpose:** Run a red to green loop that produces tests worth keeping.
 - **Use when:** Building a feature or fixing a bug test-first. Called by `/implement`, or invoked on its own for a concrete behaviour without a full specification.
 - **What a good test is:** It verifies behaviour through public interfaces, not implementation details. It reads like a specification. It survives refactors, because the implementation can change entirely while the test does not.
 - **Seams:** A seam is the public boundary you test at. Tests live at seams and never against internals. Write down the seams under test and confirm them before writing any test. No test is written at an unconfirmed seam. You cannot test everything, and agreeing seams up front is how effort lands on critical paths instead of every edge case.
 - **Anti-patterns:**
 - *Implementation-coupled.* Mocks internal collaborators, tests private methods, or verifies through a side channel such as querying the database instead of using the interface. The tell is a test that breaks on refactor while behaviour is unchanged.
 - *Tautological.* The assertion recomputes the expected value the same way the code does, so it passes by construction and can never disagree with the code. Expected values must come from an independent source: a known-good literal, a worked example, the specification.
 - *Horizontal slicing.* Writing all tests first, then all implementation. Bulk tests verify imagined behaviour, go insensitive to real changes, and commit you to a test structure before you understand the implementation.
 - **Rules of the loop:** Red before green. One seam, one test, one minimal implementation per cycle. Refactoring belongs to the review stage, not the red to green cycle.
 - **Bundled files:** `tests.md` for good and bad examples, `mocking.md` for mocking guidance, which limits mocks to system boundaries.
 - **Reads:** `CONTEXT.md`, so test names and interface vocabulary match the project's domain language.
-

6.1.11 code-review

- **Invocation:** `/code-review`
 - **Mode:** Model-invoked
 - **Path:** `skills/engineering/code-review/SKILL.md`
 - **Purpose:** Review the diff since a fixed point along two independent axes.
 - **Use when:** Before committing, before merging, or whenever you want a branch or pull request reviewed. Called automatically by `/implement`.
 - **The two axes:** Standards asks whether the code follows this repository's documented standards. Spec asks whether the code faithfully implements the originating issue or specification.
 - **How it works:** Both axes run as parallel sub-agents so neither pollutes the other's context, then the findings get reported side by side. The fixed point is whatever you supply: a commit, a branch, a tag, or a merge base. The skill confirms the reference resolves and the diff is not empty before spawning anything, so a bad reference fails early rather than inside two sub-agents. The comparison uses three dots, so it runs against the merge base.
 - **Finding the specification:** Issue references in commit messages first, then a path you passed, then a specification file matching the branch or feature, then asking you. If there is none, the Spec axis reports that and skips.
 - **Smell baseline:** On top of whatever the repository documents, the Standards axis carries a fixed set of code smells from Martin Fowler's *Refactoring*. A documented repository standard always overrides the baseline. Every smell is a labelled heuristic, never a hard violation, and anything tooling already enforces is skipped.
 - **The baseline smells:** Mysterious Name, Duplicated Code, Feature Envy, Data Clumps, Primitive Obsession, Repeated Switches, Shotgun Surgery, Divergent Change, Speculative Generality, Message Chains, Middle Man, Refused Bequest.
-

6.1.12 diagnosing-bugs

- **Invocation:** `/diagnosing-bugs`
- **Mode:** Model-invoked
- **Path:** `skills/engineering/diagnosing-bugs/SKILL.md`
- **Purpose:** Work hard bugs and performance regressions through a gated loop.
- **Use when:** Something is broken, throwing, failing, or slow, and a first glance did not solve it.
- **Redaction rule:** The skill shows commands, outputs, and captured artifacts, so every secret gets replaced with a redaction marker first. Loops get built against environment variables so credentials stay in the environment. Captured artifacts carry authorisation headers, so only the lines carrying signal get quoted. If the redacted output is not enough to diagnose, the skill says so and asks.
- **Phase 1, the heart of it:** Build a feedback loop that goes red on this specific bug. With one, bisection, hypothesis-testing, and instrumentation all follow. Without one, staring at code will not help.

- **Ways to build a loop, in rough order:** a failing test at whatever seam reaches the bug, a curl or HTTP script against a running server, a CLI invocation diffed against a known-good snapshot, a headless browser script, a replayed captured trace, a throwaway harness exercising the path with one function call, a property or fuzz loop for intermittent wrong output, a bisection harness, a differential loop comparing two versions or configurations, and as a last resort a human-in-the-loop bash script that drives the human so the loop stays structured.
- **Tighten the loop:** Treat it as a product. Make it faster by caching setup and narrowing scope. Make the signal sharper by asserting on the specific symptom rather than the absence of a crash. Make it deterministic by pinning time, seeding randomness, isolating the filesystem, and freezing the network.
- **Non-deterministic bugs:** The goal is not a clean reproduction but a higher reproduction rate. Loop the trigger a hundred times, parallelise, add stress, narrow timing windows, inject sleeps. A bug that flakes half the time is debuggable. One that flakes one time in a hundred is not.
- **When no loop is possible:** Stop and say so. List what was tried. Ask for environment access, a redacted captured artifact, or permission to add temporary instrumentation. Do not hypothesise without a loop.
- **Completion criterion for phase 1:** One named command, already run at least once with its output shown, that goes red on this bug.
- **Bundled files:** `scripts/hitl-loop.template.sh`.
- **Hands off to:** `/improve-codebase-architecture`, when the real finding is that no good seam exists to lock the bug down.

6.1.13 codebase-design

- **Invocation:** `/codebase-design`
- **Mode:** Model-invoked
- **Path:** `skills/engineering/codebase-design/SKILL.md`
- **Purpose:** Supply the shared vocabulary and principles for designing deep modules.
- **Use when:** Designing or improving an interface, deciding where a seam belongs, making code more testable or more navigable for an agent, or whenever another skill needs the vocabulary.
- **Contents:** The glossary in Section 4.3 of this document, the deep versus shallow comparison, the design tests in Section 4.4, and guidance on designing for testability.
- **Designing for testability:** Accept dependencies rather than creating them. Return results rather than producing side effects. Keep the surface small, because fewer methods means fewer tests and fewer parameters means simpler setup.
- **Relationships:** A module has exactly one interface. Depth is a property of a module measured against its interface. A seam is where the interface lives. An adapter sits at a seam and satisfies the interface. Depth produces leverage for callers and locality for maintainers.
- **Rejected framings:** Depth as a ratio of implementation lines to interface lines, because it rewards padding the implementation. Interface as only a language keyword or a class's public methods, because that is too narrow. Boundary as a synonym for seam, because it collides with domain-driven design.

- **Bundled files:** `DEEPENING.md` on deepening a cluster given its dependencies, and `DESIGN-IT-TWICE.md` on spinning up parallel sub-agents to design an interface several radically different ways before comparing them.

6.1.14 domain-modeling

- **Invocation:** `/domain-modeling`
- **Mode:** Model-invoked
- **Path:** `skills/engineering/domain-modeling/SKILL.md`
- **Purpose:** Actively build and sharpen the project's domain model as you design.
- **Use when:** Discussing terminology, writing or editing `CONTEXT.md`, or recording an ADR. Note that merely reading `CONTEXT.md` for vocabulary is not this skill. This skill is for changing the model, not consuming it.
- **What it does during a session:**
 - *Challenges against the glossary.* When a term conflicts with existing language, it says so immediately.
 - *Sharpens fuzzy language.* When a term is vague or overloaded, it proposes a precise canonical term.
 - *Discusses concrete scenarios.* It invents edge cases that force precision about where one concept ends and another begins.
 - *Cross-references with code.* When your account of how something works contradicts the code, it surfaces the contradiction.
 - *Updates inline.* Resolved terms go into `CONTEXT.md` immediately, never batched.
- **File discipline:** `CONTEXT.md` is a glossary and nothing else. No implementation detail, no specification, no scratch notes. Files get created lazily, only when there is something to write.
- **ADR test:** Offer one only when the decision is hard to reverse, surprising without context, and the result of a real trade-off. If any of the three is missing, skip it.
- **Bundled files:** `CONTEXT-FORMAT.md`, `ADR-FORMAT.md`.

6.1.15 prototype

- **Invocation:** `/prototype`
- **Mode:** Model-invoked
- **Path:** `skills/engineering/prototype/SKILL.md`
- **Purpose:** Build throwaway code that answers one design question.
- **Use when:** A question cannot be settled in conversation because it needs to be seen or driven.
- **Two branches:** For "does this logic or state model feel right", build a single shareable HTML file with free-play controls and tabbed guided walkthroughs, drivable by a non-developer. For "what should this look like", generate several radically different UI variations on one route, switchable by a URL parameter and a floating bar. Getting the branch wrong wastes the whole prototype.

- **Rules for both:** Throwaway from day one and named so a casual reader can tell. Trivial to run, from one command or one double-click. No persistence by default, because persistence is usually the thing being checked. No polish, no tests, no abstractions. Surface the full state after every action or variant switch.
 - **Capture:** Fold the validated decision into the real code, commit the prototype to a throwaway branch out of the main line, and leave a pointer to that branch on the implementation issue. Record the verdict and the question it settled. The main branch keeps only the decision.
 - **Bundled files:** `LOGIC.md`, `UI.md`.
 - **Pairs with:** `/handoff` in both directions, because a prototype lives in its own directory.
-

6.1.16 research

- **Invocation:** `/research`
 - **Mode:** Model-invoked
 - **Path:** `skills/engineering/research/SKILL.md`
 - **Purpose:** Investigate a question against high-trust primary sources and capture the findings in the repository.
 - **Use when:** A topic needs researching, API or documentation facts need gathering, or reading legwork can be delegated.
 - **How it works:** Spins up a background agent so you keep working while it reads. The agent investigates against primary sources, meaning official documentation, source code, specifications, and first-party APIs, rather than secondary write-ups. Every claim gets followed back to the source that owns it. Findings go into a single Markdown file with each claim cited, saved where the repository already keeps such notes.
-

6.1.17 resolving-merge-conflicts

- **Invocation:** `/resolving-merge-conflicts`
 - **Mode:** Model-invoked
 - **Path:** `skills/engineering/resolving-merge-conflicts/SKILL.md`
 - **Purpose:** Work an in-progress merge or rebase conflict to completion.
 - **Use when:** A merge or rebase is already in progress and conflicted.
 - **The five steps:** See the current state of the operation and the conflicting files. Find the primary sources for each side, meaning commit messages, pull requests, and original issues, and understand deeply why each change was made. Resolve each hunk, preserving both intents where possible, and where they are incompatible pick the one matching the merge's stated goal and note the trade-off. Discover the project's automated checks and run them, typically typecheck, then tests, then format. Finish the operation, staging and committing, continuing a rebase until every commit is rebased.
 - **Hard rules:** Never invent new behaviour. Always resolve. Never abort.
-

6.1.18 wizard

- **Invocation:** `/wizard`
- **Mode:** Model-invoked
- **Path:** `skills/engineering/wizard/SKILL.md`
- **Purpose:** Generate an interactive bash wizard that walks a human through steps only a human can perform.
- **Use when:** Provisioning infrastructure, setting up credentials or CI secrets, navigating an unfamiliar third-party dashboard, or running a one-off migration or cutover. Not for steps the agent can perform itself.
- **What the template already solves:** Stage-by-stage progress, confirmation gates, cross-platform URL opening including WSL, hidden entry for secrets, idempotent updates to `.env`, writes to GitHub secrets and variables, and a closing summary. The library above the stages marker is identical in every wizard, and that consistency is the point. Never hand-edit it.
- **The four steps:** Scope the procedure by reading the repository first, including environment files, README, compose files, framework configuration, and every workflow reference to a secret or variable. Map each stage's journey as the precise path a human follows, and where the current interface is unknown, ask rather than invent. Author the wizard by copying the template and writing one stage per step in dependency order. Verify and hand off.
- **Standards to hold:** Open the URL before asking for its value. Use hidden entry for anything secret. Persist every captured value. Set only the secrets CI actually needs. Confirm before any irreversible action. Keep each stage to one focused task, because each stage clears the screen.
- **Lifetime:** Ephemeral by default, saved to a scratch path and deleted afterwards. Commit it only when the setup path should live in the repository.
- **Bundled files:** `template.sh`.

6.2 Productivity skills

Seven skills for general workflow, not specific to code. All are included in the plugin.

6.2.1 grill-me

- **Invocation:** `/grill-me`
- **Mode:** User-invoked
- **Path:** `skills/productivity/grill-me/SKILL.md`
- **Purpose:** Get relentlessly interviewed about a plan or design until every branch of the design tree is resolved.
- **Use when:** You have an idea and no repository to leave a trail in, or you want the interview without the documentation side effects.
- **How it works:** It is a one-line skill that runs the `/grilling` session. The discipline lives in `/grilling`.

- **Note:** This is the skill from the talk. Two lines of instruction turn the agent into a productive adversary that will ask forty, sixty, sometimes a hundred questions before it is satisfied that understanding is shared.
-

6.2.2 grilling

- **Invocation:** `/grilling`
 - **Mode:** Model-invoked
 - **Path:** `skills/productivity/grilling/SKILL.md`
 - **Purpose:** The reusable interview primitive behind `grill-me`, `grill-with-docs`, `triage`, `wayfinder`, and `improve-codebase-architecture`.
 - **Use when:** Any plan, decision, or idea needs stress-testing.
 - **How it works:** The plan is mapped as a design tree, where every decision branches into the decisions that hang off it. Work proceeds in rounds. The frontier is every decision whose prerequisites are settled, meaning the questions answerable now without guessing. The whole frontier goes out in one round, numbered, each question carrying a recommended answer. Then it waits.
 - **Question format:** A numbered question with a title, a body that may run to several paragraphs and may offer choices, followed by the recommended answer.
 - **Round mechanics:** Each round of answers reshapes the tree. Settled decisions push the frontier outward and unblock questions that depended on them. A question whose answer depends on another question still open belongs to a later round, not this one.
 - **Division of labour:** Finding facts is the agent's job, never yours. When a question needs a fact from the environment, the agent dispatches a sub-agent rather than asking you something it could look up. It does not block on that: only questions downstream of the exploration wait. The decisions are yours, and each one gets put to you.
 - **Completion:** The session ends when the frontier is empty, meaning every branch was visited and nothing was silently assumed. The agent does not act until you confirm understanding is shared.
-

6.2.3 handoff

- **Invocation:** `/handoff`
- **Mode:** User-invoked
- **Path:** `skills/productivity/handoff/SKILL.md`
- **Purpose:** Compact the current conversation into a document another agent can pick up.
- **Use when:** Crossing a session boundary, moving to a prototype directory and back, or ending a session with work still open.
- **How it works:** Writes a handoff document summarising the conversation, saved to the operating system temporary directory rather than the workspace. It includes a suggested skills section naming what the next agent should invoke. It does not duplicate content already captured in specifications, plans, ADRs, issues,

commits, or diffs, and references those by path or URL instead. It redacts secrets and personal information. If you pass an argument, it treats that as the focus of the next session and tailors the document accordingly.

6.2.4 teach

- **Invocation:** /teach
 - **Mode:** User-invoked
 - **Path:** skills/productivity/teach/SKILL.md
 - **Purpose:** Teach a skill or concept across multiple sessions, using the current directory as a stateful teaching workspace.
 - **Use when:** You want to learn something properly rather than get one explanation.
 - **Workspace files:**
 - MISSION.md, capturing why you want the topic, which grounds all teaching.
 - reference/*.html, compressed learnings such as cheat sheets, reference algorithms, syntax summaries, and glossaries, built to print well and be scanned quickly.
 - RESOURCES.md, the list of resources that ground the teaching.
 - learning-records/*.md, numbered records of what you have learned, equivalent to decision records, used to calculate the zone of proximal development.
 - lessons/*.html, the primary unit of teaching, each one self-contained and tightly scoped to one thing tied to the mission.
 - assets/*, reusable components shared across lessons.
 - NOTES.md, a scratchpad for preferences and working notes.
 - **Philosophy:** Deep learning needs knowledge captured from high-quality sources, skills acquired through interactive lessons, and wisdom from other practitioners. Before RESOURCES.md is well populated, the priority is finding high-quality resources. Parametric knowledge is never trusted. The balance shifts by topic: theoretical physics leans toward knowledge, physical practice leans toward skills.
 - **Two kinds of strength:** Fluency strength is in-the-moment retrieval. Storage strength is long-term retention. The skill treats them separately.
 - **Bundled files:** MISSION-FORMAT.md, RESOURCES-FORMAT.md, LEARNING-RECORD-FORMAT.md, GLOSSARY-FORMAT.md.
-

6.2.5 to-questionnaire

- **Invocation:** /to-questionnaire
- **Mode:** User-invoked
- **Path:** skills/productivity/to-questionnaire/SKILL.md
- **Purpose:** Turn a decision you cannot make alone into a questionnaire for the one person who can.
- **Use when:** Someone else holds knowledge you need, and you want it either async or in one meeting.

- **Central move:** Grill the send, not the subject. The interview covers only what you can always answer: who it goes to, and what you need back. The questions in the document then target the gap between what the recipient knows and what you need.
- **Three steps:** Establish who the recipient is, their role, expertise, and relationship to you, which fixes tone and how much context the document must carry. Establish what you need back as a concrete list. Write the questionnaire to a file in the current directory and report the path.
- **Document structure:** A purpose line naming the decision riding on it, a from and to line explaining how answers will be used, one paragraph of context, a note on deadline and effort making clear that partial answers and admissions of uncertainty are useful, then themed sections. Questions run most important first, because async may give you only one pass. Every question is one idea, never compound, with an answer stub beneath it.

6.2.6 wait-what

- **Invocation:** `/wait-what`
- **Mode:** User-invoked
- **Path:** `skills/productivity/wait-what/SKILL.md`
- **Purpose:** Make the agent re-pitch a message that did not land.
- **Use when:** The moment you lose the thread. Immediately, not three messages later.
- **How it works:** A single instruction. The agent stops, adds the context you were missing, and re-pitches in simplified technical English, using the vocabulary from `CONTEXT.md`.
- **Why it works:** The re-pitch is anchored to your project glossary, so the explanation lands in terms you already own rather than generic phrasing.

6.2.7 writing-for-agents

- **Invocation:** `/writing-for-agents`
- **Mode:** Model-invoked
- **Path:** `skills/productivity/writing-for-agents/SKILL.md`
- **Purpose:** Reference for writing any document an agent consumes.
- **Use when:** Creating or editing a skill, or modifying `AGENTS.md` or `CLAUDE.md`.
- **Core claim:** The packaging differs but the writing does not. The same levers make each document predictable, meaning the agent takes the same process every run, not that it produces the same output.
- **Context pointers:** A pointer does two jobs: state what the material is, and list the branches that should trigger reaching it. A must-have target behind a weakly worded pointer is a variance bug, so sharpen the wording before inlining the material. Rules: front-load the leading word, use one trigger per branch and collapse synonyms, and cut identity the body already carries.
- **The two budgets:** Context load is what always-loaded material costs the agent's window every turn. Cognitive load is what it costs the human to know which documents exist. Cognitive load is not to be

minimised blindly, because it is the price of human agency. Spend it where judgement matters and remove it where it does not.

- **Information hierarchy:** Documents mix steps, meaning ordered actions, and reference, meaning facts consulted on demand. Three tiers: in-file step, in-file reference, and disclosed reference behind a pointer. Push too little down and the top bloats. Push too much and you hide what the agent needs.
 - **The disclosure test:** Inline what every branch needs. Push behind a pointer what only some branches reach.
 - **Bundled files:** `SKILL-MECHANICS.md`, covering frontmatter, the invocation choice, and router skills.
-

6.3 Miscellaneous skills

Four tools kept around but rarely used. Not promoted in the plugin. Install individually if wanted.

6.3.1 git-guardrails-claude-code

- **Path:** `skills/misc/git-guardrails-claude-code/SKILL.md`
 - **Mode:** Model-invoked
 - **Purpose:** Install hooks that block dangerous git commands before they execute.
 - **What gets blocked:** `git push` in all variants including `force`, `git reset --hard`, `git clean -f` and `-fd`, `git branch -D`, and `git checkout .` or `git restore ..`
 - **Bundled files:** `scripts/block-dangerous-git.sh`.
-

6.3.2 setup-pre-commit

- **Path:** `skills/misc/setup-pre-commit/SKILL.md`
 - **Mode:** Model-invoked
 - **Purpose:** Set up commit-time formatting, typechecking, and tests.
 - **What it installs:** A Husky pre-commit hook, lint-staged running Prettier over staged files, a Prettier configuration if one is missing, and typecheck and test scripts wired into the hook.
-

6.3.3 migrate-to-shoehorn

- **Path:** `skills/misc/migrate-to-shoehorn/SKILL.md`
 - **Mode:** Model-invoked
 - **Purpose:** Migrate test files from type assertions to `@total-typescript/shoehorn`.
 - **Why:** Shoehorn allows partial test data while keeping the type checker satisfied, replacing assertions with type-safe alternatives.
 - **Hard rule:** Test code only. Never use it in production code.
-

6.3.4 scaffold-exercises

- **Path:** `skills/misc/scaffold-exercises/SKILL.md`
- **Mode:** Model-invoked
- **Purpose:** Create exercise directory structures with sections, problems, solutions, and explainers that pass linting.
- **Naming convention:** Sections are numbered directories inside `exercises/`. Exercises are numbered within a section using a section and exercise number. Names are lowercase with hyphens.

6.4 In-progress skills

Six beta skills, public on purpose so they can be tested. Excluded from the plugin and from the main readme until they graduate. They get no documentation pages and can change or disappear without warning.

Install one directly:

```
npx skills@latest add mattpocock/skills --skill=<name>
```

6.4.1 loop-me

- **Path:** `skills/in-progress/loop-me/SKILL.md`
- **Mode:** User-invoked
- **Purpose:** Grill yourself into implementable workflow specifications over multiple sessions, using the current directory as a stateful workspace.
- **The loop lens:** A loop is a recurring pattern in your life: a career, a week, a morning, a single repeated activity. Picturing a life as loops within loops reveals how predictable its activities are, which is what makes them worth delegating. A workflow is the specification of one loop made real, and workflows live in `workflows/*.md` as the source of truth.
- **Vocabulary:** A trigger is what fires each run, either an event or a schedule, with event-triggering usually more efficient. A checkpoint is a human-in-the-loop point for verification or decision, and some workflows have none. Push right means deferring the checkpoint as far as it will go, doing maximal work before involving the human so they are asked once, late, with everything prepared.
- **Constraint:** Mandate nothing structural. A workflow needs no AI, no checkpoint, and no schedule unless the interview shows it does.

6.4.2 claude-handoff

- **Path:** `skills/in-progress/claude-handoff/SKILL.md`
 - **Mode:** User-invoked
 - **Purpose:** Hand the conversation to a fresh background agent that picks up immediately.
 - **How it differs from `/handoff`:** Instead of saving a document, it launches a background agent seeded with the summary as its prompt, starting in the current working directory and returning immediately.
 - **Requirement:** Always pass a descriptive name, because it sets the display name in the job list, session picker, and terminal title.
 - **Same disciplines as `/handoff`:** A suggested skills section, no duplication of content already captured elsewhere, and redaction of secrets, which matters more here because the summary becomes the agent's prompt.
-

6.4.3 setup-ts-deep-modules

- **Path:** `skills/in-progress/setup-ts-deep-modules/SKILL.md`
 - **Mode:** User-invoked
 - **Purpose:** Make every package in a TypeScript repository a deep module, enforced by tooling.
 - **The shape enforced:** A package's public surface is its entry-point files at the package root. Everything in subfolders is hidden. Implementation lives in `lib/` and is free to import itself. Tests are co-located in a subfolder, which makes them private. A package may expose several entry points.
 - **How:** Installs dependency-cruiser and the rules that make entry points the only way in, then proves the rules bite.
 - **Bundled files:** `dependency-cruiser.config.cjs`.
 - **Reads vocabulary from:** `/codebase-design`.
-

6.4.4 writing-fragments

- **Path:** `skills/in-progress/writing-fragments/SKILL.md`
 - **Mode:** User-invoked
 - **Purpose:** Mine you for fragments and collect them as raw material.
 - **Stage:** Pure explore. Widen the space of what could be written without committing to structure. Imposing phases, outlines, or article structure is out of scope.
 - **How it works:** Runs a grilling session about whatever you want to write about. As fragments emerge from either side of the conversation, they get appended to one Markdown file. Capture starts from the very first thing you say, including the opening prompt. The file opens with a single working title and nothing else: no metadata, no table of contents, no date.
-

6.4.5 writing-shape

- **Path:** `skills/in-progress/writing-shape/SKILL.md`
- **Mode:** User-invoked
- **Purpose:** Shape a pile of raw material into an article, paragraph by paragraph.
- **Stage:** Exploit. The exploring is done and the pile is fixed, so commit to a structure and mine the pile to fill it.
- **How it works:** Reads the input file end to end first. The input file is read-only to this skill, and the article is written separately. Format of the input does not matter: a tidy list, a wall of prose, or a transcript.
- **Grounding:** Settle what the reader knows walking in. Everything else must be grounded by an earlier block before a later block can lean on it.

6.4.6 writing-beats

- **Path:** `skills/in-progress/writing-beats/SKILL.md`
- **Mode:** User-invoked
- **Purpose:** Assemble raw material into a journey of beats.
- **Stage:** Exploit, run as a choose-your-own-adventure.
- **The loop:** Establish the prerequisites, meaning what the audience already knows. Offer two or three candidate starting beats, each a different entry point, each leaning only on grounded concepts and each noting what new concepts it grounds. You pick one. Only that beat gets written to the article file, then it stops. It re-reads the article from disk and offers the next candidates, each reachable from the current grounded set. Repeat until the article reaches a natural end.
- **Beat length:** Whatever that beat naturally is, from one sentence to several paragraphs.

7. Bundled Reference Files

Files loaded only when the skill that owns them needs them. This is progressive disclosure in practice.

File	Owner skill	Contents
PHASE - BOUNDARIES .md	ask-matt	Where compaction belongs, and why mid-phase compaction loses the thread
AGENT - BRIEF .md	triage	How to write the durable brief that becomes the contract for an agent
OUT - OF - SCOPE .md	triage	How the rejection knowledge base preserves reasoning and deduplicates requests

File	Owner skill	Contents
HTML-REPORT.md	improve-codebase-architecture	Report scaffold, diagram patterns, styling
tests.md	tdd	Worked examples of good and bad tests
mocking.md	tdd	Mock at system boundaries only
DEEPENING.md	codebase-design	Dependency categories, seam discipline, replace rather than layer
DESIGN-IT-TWICE.md	codebase-design	Parallel sub-agents design several interfaces, then compare
CONTEXT-FORMAT.md	domain-modeling	The glossary file format
ADR-FORMAT.md	domain-modeling	Numbered decision record format
LOGIC.md	prototype	The shareable single-file state prototype
UI.md	prototype	Multiple switchable UI variations on one route
SKILL-MECHANICS.md	writing-for-agents	Frontmatter, invocation choice, router skills
MISSION-FORMAT.md	teach	Why the learner wants the topic
RESOURCES-FORMAT.md	teach	The grounding resource list
LEARNING-RECORD-FORMAT.md	teach	Numbered records of what was learned
GLOSSARY-FORMAT.md	teach	Topic glossary format
template.sh	wizard	The wizard library and stage scaffold
hitl-loop.template.sh	diagnosing-bugs	Structured human-in-the-loop feedback loop
block-dangerous-git.sh	git-guardrails-claude-code	The blocking hook
dependency-cruiser.config.cjs	setup-ts-deep-modules	Entry-point enforcement rules
issue-tracker-github.md	setup-matt-pocock-skills	GitHub workflow via the gh CLI
issue-tracker-gitlab.md	setup-matt-pocock-skills	GitLab workflow via the glab CLI
issue-tracker-local.md	setup-matt-pocock-skills	Local markdown tracker convention

File	Owner skill	Contents
trriage-labels.md	setup-matt-pocock-skills	Canonical role to label mapping
domain.md	setup-matt-pocock-skills	Where domain documents live

8. Artifacts These Skills Create In Your Repository

Artifact	Created by	Purpose	Rules
CONTEXT.md	domain-modeling	The project glossary	Glossary only. No implementation detail, no specifications, no scratch notes. Created lazily on the first resolved term.
CONTEXT-MAP.md	domain-modeling	Points at each context in a multi-context repository	Present only when more than one bounded context exists
docs/adr/NNNN-slug.md	domain-modeling	Decision records	Sequential numbering. Created lazily. Written only when the three-part test passes.
docs/agents/*	setup-matt-pocock-skills	Per-repository configuration	Issue tracker workflow, triage label mapping, domain document layout
.scratch/	to-tickets, to-spec	Local markdown issue tracker	Used only when you chose the local tracker
.out-of-scope/	trriage	Rejection knowledge base	Preserves why something was rejected, and deduplicates repeat requests
Architecture report	improve-codebase-architecture	The review output	Written to the temporary directory, never the repository
Handoff document	handoff	Session continuity	Written to the temporary directory, never the workspace
Prototype branch	prototype	The captured throwaway	Committed out of the main line, referenced from the implementation issue

9. Playbooks

9.1 A new feature in an existing repository

```
1. /grill-with-docs      interview until the frontier is empty
2. /to-spec              synthesise the conversation, confirm seams
3. /to-tickets           vertical slices with blocking edges
   -- clear context --
4. /implement            one ticket, fresh context, tdd inside
   -- clear context --
5. /implement            next ticket, repeat
```

Keep steps 1 to 3 in one context window.

9.2 A small change

```
1. /grill-with-docs      shorter interview, same discipline
2. /implement            same context window, no spec needed
```

9.3 A hard bug

```
1. /diagnosing-bugs      build a loop that goes red on this bug
2. ...                   minimise, hypothesise, instrument, fix
3. ...                   add the regression test
4. /improve-codebase-architecture  only if the finding was "no good seam"
```

9.4 A question that needs to be seen

```
1. /grill-with-docs      until a question needs a runnable answer
2. /handoff              out to a prototype directory
3. /prototype            throwaway code, one question
4. /handoff              back with what was learned
5. /to-spec              fold the decision in
```

9.5 A large, foggy effort

- | | |
|---------------------------------------|---------------------------------------|
| 1. <code>/wayfinder</code> | name the destination, chart the map |
| 2. ... | resolve one decision ticket at a time |
| 3. ... | until nothing is left to decide |
| 4. <code>/to-spec, /to-tickets</code> | hand off to the main flow |

9.6 Incoming reports

- | | |
|----------------------------|---|
| 1. <code>/triage</code> | categorise, verify, grill if needed, write briefs |
| 2. <code>/implement</code> | pick up anything marked ready-for-agent |

9.7 Routine maintenance

```
every few days: /improve-codebase-architecture
before merging: /code-review
when lost:      /wait-what
when unsure:    /ask-matt
```

10. Practice Notes

10.1 Habits worth keeping

- Interview before building, every time you make a change.
- Keep the interview, the specification, and the tickets in one context window.
- Start each implementation in a fresh context.
- Confirm the seams before writing any test.
- Update the glossary the moment a term resolves, never in a batch.
- Write a decision record only when all three conditions hold.
- Run the architecture survey on a schedule rather than on a crisis.
- Compact at phase boundaries, not in the middle of work.

10.2 Mistakes to avoid

- Installing both the plugin and the copied files, which duplicates every skill.
- Skipping the setup skill, which leaves later skills without a tracker.
- Triaging tickets that `/to-tickets` already produced.

- Testing past an interface, which usually means the module is the wrong shape.
- Introducing a seam where nothing varies across it.
- Writing all tests first, then all implementation.
- Treating `CONTEXT.md` as a specification or a scratchpad.
- Reaching for wayfinding on a well-scoped feature.
- Hypothesising about a bug before a loop goes red on it.
- Aborting a merge rather than resolving it.

10.3 Adaptation

These are small documents by design. Fork them, cut what does not apply, and rename things to match your own vocabulary. The route that copies editable files into your project exists precisely so you can do this. The only part worth preserving carefully is the vocabulary, because its value comes from being used consistently.

11. Source Library

The skills draw on these works. Reading any of them makes the skills easier to adapt.

Work	Author	What the skills take from it
A Philosophy of Software Design	John Ousterhout	Complexity as resistance to change. Deep versus shallow modules. Design it twice.
The Pragmatic Programmer	David Thomas and Andrew Hunt	Nobody knows exactly what they want. Software entropy. The rate of feedback is your speed limit. Tracer bullets.
The Design of Design	Frederick P. Brooks	The design concept. The design tree.
Domain-Driven Design	Eric Evans	Ubiquitous language. Bounded contexts.
Extreme Programming Explained	Kent Beck	Invest in the design of the system every day.
Refactoring	Martin Fowler	The code smell baseline used by the review skill.
Working Effectively with Legacy Code	Michael Feathers	The seam.

12. Links

Resource	URL
Repository	https://github.com/mattpocock/skills
The talk this document opens with	https://youtu.be/v4F1gFy-hqg
Newsletter	https://www.aihero.dev/s/skills-newsletter
Author site	https://www.aihero.dev
Installer index	https://skills.sh/mattpocock/skills
Claude Code plugin documentation	https://code.claude.com/docs/en/plugins
Dependency-cruiser	https://github.com/sverweij/dependency-cruiser
GitLab CLI	https://gitlab.com/gitlab-org/cli

13. Attribution And License

The skills, their text, and the arguments summarised in Section 1 are the work of Matt Pocock, published at github.com/mattpocock/skills under the MIT License, copyright 2026 Matt Pocock.

The MIT License grants permission to use, copy, modify, merge, publish, distribute, sublicense, and sell copies of the software, on the condition that the copyright notice and permission notice are included in all copies or substantial portions. The software is provided as is, without warranty of any kind.

This document is a derived instructional reference. It summarises and reorganises the repository contents. For the authoritative text of any skill, read its `SKILL.md` in the repository.

14. Appendix: Full Skill Index

#	Skill	Bucket	Mode	In plugin
1	ask-matt	engineering	User	Yes
2	code-review	engineering	Model	Yes

#	Skill	Bucket	Mode	In plugin
3	codebase-design	engineering	Model	Yes
4	diagnosing-bugs	engineering	Model	Yes
5	domain-modeling	engineering	Model	Yes
6	grill-with-docs	engineering	User	Yes
7	implement	engineering	User	Yes
8	improve-codebase-architecture	engineering	User	Yes
9	prototype	engineering	Model	Yes
10	research	engineering	Model	Yes
11	resolving-merge-conflicts	engineering	Model	Yes
12	setup-matt-pocock-skills	engineering	User	Yes
13	tdd	engineering	Model	Yes
14	to-spec	engineering	User	Yes
15	to-tickets	engineering	User	Yes
16	triage	engineering	User	Yes
17	wayfinder	engineering	User	Yes
18	wizard	engineering	Model	Yes
19	grill-me	productivity	User	Yes
20	grilling	productivity	Model	Yes
21	handoff	productivity	User	Yes
22	teach	productivity	User	Yes
23	to-questionnaire	productivity	User	Yes
24	wait-what	productivity	User	Yes
25	writing-for-agents	productivity	Model	Yes
26	git-guardrails-claude-code	misc	Model	No
27	migrate-to-shoehorn	misc	Model	No
28	scaffold-exercises	misc	Model	No

#	Skill	Bucket	Mode	In plugin
29	setup-pre-commit	misc	Model	No
30	claude-handoff	in-progress	User	No
31	loop-me	in-progress	User	No
32	setup-ts-deep-modules	in-progress	User	No
33	writing-beats	in-progress	User	No
34	writing-fragments	in-progress	User	No
35	writing-shape	in-progress	User	No

End of document.